

APACHE HADOOP

Car Sales Dataset
HDFS Storage and Data Operations

Student Name	Jeevitha V
USN	4GW23CI020
Practical	Apache Hadoop – Car Sales Dataset
Hadoop Version	3.4.3
Environment	Ubuntu Linux

MLOps Practical Assignment

1. Aim

To store a Car Sales CSV dataset in Hadoop Distributed File System (HDFS) and perform basic data operations such as displaying, filtering, counting, sorting and retrieving the file.

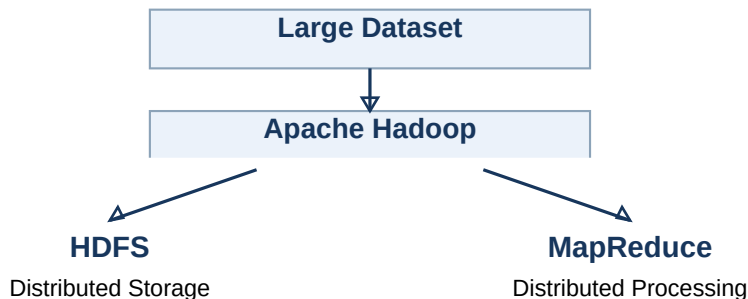
2. Problem Statement

A car sales dataset contains details such as car ID, brand, model, manufacturing year, price, engine capacity and city. The task is to store this CSV dataset in HDFS and demonstrate basic operations on the stored data.

3. What is Apache Hadoop?

Apache Hadoop is an open-source framework used for distributed storage and processing of large datasets. It allows data to be stored and processed across multiple computers instead of relying on a single machine.

In this practical, the main component demonstrated is HDFS, which stores and provides access to the Car Sales CSV file.



4. Why do we use Hadoop?

- **Distributed storage:** HDFS can store data across multiple machines.
- **Scalability:** Additional machines can be added as the dataset grows.
- **Fault tolerance:** HDFS can maintain replicas of data blocks to help recover from machine failures.
- **Big data processing:** Hadoop includes processing tools such as MapReduce.

5. Hadoop Components

Hadoop includes storage and processing components. This practical mainly demonstrates HDFS.

5.1 HDFS – Hadoop Distributed File System

HDFS is Hadoop's storage system. It stores files in a distributed manner and allows users to access them through Hadoop file-system commands.

5.2 NameNode

The NameNode manages file-system metadata, including file names, directories, permissions and the locations of data blocks.

5.3 DataNode

The DataNode stores the actual data blocks of files in HDFS.

5.4 MapReduce

MapReduce is Hadoop's data-processing model. In this practical, grep, awk, sort and wc are Linux command-line utilities used after reading the file from HDFS. A separate MapReduce program is not implemented.

6. Software and Technologies Used

Technology	Purpose
Ubuntu Linux	Operating system used for the practical.
Java	Required runtime environment for Hadoop.
Apache Hadoop 3.4.3	Framework used for distributed storage and processing.
HDFS	Stores and provides access to the Car Sales CSV dataset.
Linux commands	Used for file creation, display, filtering, counting and sorting.

7. Hadoop Version and Installation Verification

Open Ubuntu Terminal. Check the installed Hadoop and Java versions using the following commands:

```
hadoop version  
java -version
```

The first command displays Hadoop version and build information. The second command displays the installed Java runtime version. Java is required for Hadoop to run.

8. Complete Practical Demonstration – Step by Step

PART A – Start Hadoop

Step 1: Open Ubuntu Terminal and go to the home directory

```
cd ~
```

cd means Change Directory. The symbol ~ represents the home directory of the current Ubuntu user. This command takes us to that directory, where we can create and manage practical files.

Check the Hadoop installation:

```
hadoop version
```

This displays the installed Hadoop version. For this setup, the version used is Hadoop 3.4.3.

Step 2: Start HDFS

```
start-dfs.sh
```

This script starts the main HDFS services, including the NameNode, DataNode and SecondaryNameNode.

Check whether Hadoop is running:

```
jps
```

jps means Java Process Status. It displays Java processes currently running. Expected entries may include NameNode, DataNode, SecondaryNameNode and Jps. If these Hadoop processes are already running, do not start HDFS again.

PART B – Create the Car Sales Dataset

Step 3: Create the CSV file

```
nano car_sales.csv
```

nano is a simple terminal-based text editor. Enter the following dataset in the editor:

```
Car_ID,Brand,Model,Year,Price,Liters,City
C001,Toyota,Fortuner,2022,3800000,2.8,Bangalore
C002,Hyundai,Creta,2023,1800000,1.5,Mysore
C003,Maruti,Baleno,2021,900000,1.2,Mangalore
C004,Tata,Nexon,2022,1400000,1.5,Hubli
C005,Honda,City,2023,1600000,1.5,Bangalore
C006,Kia,Seltos,2022,1900000,1.5,Mysore
C007,Maruti,Swift,2020,750000,1.2,Bangalore
C008,Tata,Harrier,2023,2200000,2.0,Mangalore
C009,Toyota,Innova,2021,2700000,2.4,Hubli
C010,Hyundai,Verna,2022,1500000,1.5,Bangalore
```

Save the file using Ctrl + O, press Enter, and then press Ctrl + X to exit nano.

Verify the dataset:

```
cat car_sales.csv
```

cat displays the contents of a local file. The output should contain one header row and ten car records.

8. Complete Practical Demonstration – Continued

PART C – Create an HDFS Directory

Step 4: Create a folder in HDFS

```
hdfs dfs -mkdir -p /car_sales
```

hdfs runs the Hadoop file-system command-line interface. dfs specifies that the operation is performed on HDFS. -mkdir creates a directory. -p creates parent directories if needed and avoids an error if the directory already exists. /car_sales is the HDFS directory path.

Check the HDFS root directory:

```
hdfs dfs -ls /
```

The listing should contain the car_sales directory.

PART D – Upload the Dataset to HDFS

Step 5: Upload the CSV file

```
hdfs dfs -put car_sales.csv /car_sales/
```

-put copies a file from the local Ubuntu file system to HDFS. Here, car_sales.csv is copied into the /car_sales directory.

Verify the uploaded file:

```
hdfs dfs -ls /car_sales
```

The output should list car_sales.csv.

PART E – Perform Data Operations

Step 6: Display the complete dataset

```
hdfs dfs -cat /car_sales/car_sales.csv
```

hdfs dfs -cat displays the contents of a file stored in HDFS. This verifies that the dataset has been uploaded and can be read from Hadoop.

Step 7: Count the number of cars

```
hdfs dfs -cat /car_sales/car_sales.csv | tail -n +2 | wc -l
```

hdfs dfs -cat reads the file from HDFS. The pipe (|) sends output to the next command. tail -n +2 skips the first line (the header). wc -l counts the remaining lines.

Expected output: 10. Therefore, the dataset contains ten car records, excluding the header.

9. Perform Data Operations – Continued

Step 8: Find Toyota cars

```
hdfs dfs -cat /car_sales/car_sales.csv | grep Toyota
```

grep searches for lines containing a particular word or pattern. Here, it displays records containing the brand Toyota.

```
C001,Toyota,Fortuner,2022,3800000,2.8,Bangalore  
C009,Toyota,Innova,2021,2700000,2.4,Hubli
```

Step 9: Find cars sold in Bangalore

```
hdfs dfs -cat /car_sales/car_sales.csv | grep Bangalore
```

The grep command searches for records containing the word Bangalore. It filters the dataset based on city.

Step 10: Find cars costing more than ₹20 lakh

```
hdfs dfs -cat /car_sales/car_sales.csv | awk -F',' 'NR==1 || $5>2000000'
```

This command selects the header and records whose price is greater than ₹20,00,000.

- **awk**: A command-line utility used to process structured text.
- **-F','**: Specifies comma as the field separator.
- **NR**: Represents the current record (line) number.
- **NR==1**: Keeps the first line, which is the CSV header.
- **\$5**: Refers to the fifth column, Price.
- **\$5>2000000**: Selects records whose price is greater than 2,000,000.

Expected result: Three cars meet the condition: Toyota Fortuner, Tata Harrier and Toyota Innova.

Step 11: Sort cars according to price

```
hdfs dfs -cat /car_sales/car_sales.csv | sort -t',' -k5 -n
```

sort arranges lines using the fifth field (Price). -t',' specifies comma as the field separator, -k5 selects the fifth field and -n performs numeric sorting. The command as written also sorts the header line; it does not separately preserve the header at the top.

Step 12: Check the HDFS file size

```
hdfs dfs -du -h /car_sales/car_sales.csv
```

-du displays the disk usage of the specified HDFS file. The -h option displays the size in a human-readable format, such as KB or MB.

10. Download the Dataset from HDFS

Step 13: Copy the file back to Ubuntu

```
hdfs dfs -get /car_sales/car_sales.csv car_sales_copy.csv
```

-get copies a file from HDFS to the local Ubuntu filesystem. It is the reverse of the -put operation.

Check the local files:

```
ls
```

The directory should contain car_sales.csv and car_sales_copy.csv. The second file is the copy retrieved from HDFS.

11. Finish the Demonstration

Step 14: Check the HDFS directory

```
hdfs dfs -ls /car_sales
```

This lists the contents of the HDFS directory and verifies that car_sales.csv is still stored there.

Step 15: Stop Hadoop

```
stop-dfs.sh
```

After completing all operations, this command stops the HDFS services, including the NameNode, DataNode and SecondaryNameNode.

Wait a few seconds and check the running processes:

```
jps
```

The Hadoop processes should no longer be listed. The Jps process itself may appear because the command is currently being executed.

12. Complete Demonstration Flow

```
Car Sales Dataset
  ↓
Ubuntu System
  ↓
Create car_sales.csv
  ↓
Create /car_sales in HDFS
  ↓
Upload using -put
  ↓
Display using -cat
  ↓
Count | Filter | Sort | Check file size
  ↓
Retrieve using -get
  ↓
Stop HDFS
```

13. Important Commands and Their Meanings

Command	Purpose
cd ~	Go to the current user's home directory.
java -version	Check the installed Java version.
hadoop version	Check the installed Hadoop version.
start-dfs.sh	Start HDFS services.
jps	Check running Java/Hadoop processes.
nano car_sales.csv	Create or edit the CSV dataset.
cat car_sales.csv	Display the local CSV file.
hdfs dfs -mkdir -p /car_sales	Create the HDFS directory.
hdfs dfs -ls /	List the HDFS root directory.
hdfs dfs -put ...	Copy a local file into HDFS.
hdfs dfs -cat ...	Display a file stored in HDFS.
tail -n +2	Skip the first line of the file.
wc -l	Count lines.
grep	Search/filter lines containing a word or pattern.
awk	Process text using fields and conditions.
sort	Sort lines or records.
hdfs dfs -du -h ...	Display HDFS file size in human-readable form.
hdfs dfs -get ...	Copy a file from HDFS to local storage.
stop-dfs.sh	Stop HDFS services.

14. Result

The Car Sales CSV dataset was successfully created in Ubuntu and uploaded to the HDFS directory /car_sales. The dataset was displayed from HDFS, and basic operations were demonstrated, including counting ten car records, filtering Toyota and Bangalore records, identifying cars priced above ₹20 lakh, sorting by price and checking file size. The file was also retrieved from HDFS to the local Ubuntu filesystem using the -get command.